MATHÉMATIQUES ET INFORMATIQUE
Sciences
Université Paris Cité

Mémoire

Programmation du Plakoto

Auteurs :
YA Hassrol
GHANIME Joseph
BENNAMANE Mahdi
REN Ziyi

Encadrant :
BOUZY Bruno

Groupe : L2N1

Table des matières

1 Introduction **2**

2 Contexte **2**

3 Objectif **3**

4 Conception des méthodes **3**
4.1 ***Conception générale*** 4
4.2 ***Conception détaillée de certaines fonctions importantes*** 4

5 Optimisations majeures **5**

6 Résultats et analyse **6**
6.1 ***Statistiques des parties IA vs IA*** 6
6.2 ***Analyse des résultats des simulations Monte Carlo*** 6
6.3 ***Courbes de performance et interprétation*** 7
6.4 ***Explication des choix de ns et impact stratégique*** 8

7 Déroulement du développement **8**

8 Discussion collective **9**
8.1 ***Organisation et gestion de projet*** 9
8.2 ***Travail en équipe et communication*** 9
8.3 ***Réussites et apprentissages*** 10

9 Conclusion **10**
9.1 *Bilan et synthèse* 10
9.2 *Perspectives et améliorations* 10

1 Introduction

Dans le cadre de notre formation en deuxième année de Licence d'Informatique, nous avons été amenés à travailler sur un projet de programmation en équipe. Ce projet, mené dans le cadre de l'Unité d'Enseignement « Projet de Programmation », visait à concevoir et développer un logiciel complet en autonomie, depuis la définition des objectifs jusqu'à l'implémentation finale et la présentation des résultats.

Notre choix s'est porté sur la numérisation du **jeu de société Plakoto**, un jeu appartenant à la famille des backgammons. Ce jeu, à la fois stratégique et méconnu, nous a semblé idéal pour mettre en œuvre une variété de compétences : compréhension de règles complexes, modélisation algorithmique, programmation orientée objet, gestion d'une interface utilisateur, développement d'une intelligence artificielle, et optimisation des performances.

Ce projet nous a permis d'aborder toutes les étapes du cycle de vie d'un logiciel :

— Analyse du cahier des charges,
— Conception de l'architecture,
— Développement de fonctionnalités,
— Phase de test,
— Mise en place d'une IA Monte Carlo,
— Simulation de parties,
— Et analyse statistique des performances.

En parallèle des aspects techniques, une attention particulière a été portée à la **gestion de projet**, à la **répartition des rôles**, à l'utilisation d'outils collaboratifs comme les systèmes de versionnage, ainsi qu'à la **communication interpersonnelle**.

Ainsi, ce projet constitue une **expérience complète** combinant rigueur algorithmique, travail en équipe, structuration logicielle et approfondissement de notre maîtrise du langage Python. Il nous a également donné l'opportunité d'explorer des domaines avancés comme les **simulations aléatoires**, le **traitement parallèle**, et l'**optimisation heuristique**, tout en assurant la qualité et la maintenabilité du code.

2 Contexte

Ce projet s'inscrit dans le cadre de l'unité d'enseignement « Projet de Programmation », composante essentielle de notre deuxième année de Licence d'Informatique. Il s'agit d'un travail en équipe visant à concevoir et développer une application logicielle complète, en autonomie, tout en respectant un cahier des charges pouvant être imposé ou défini par les étudiants eux-mêmes.

L'objectif principal de cette unité est de mettre en pratique les compétences acquises dans les domaines de la conception logicielle, de la programmation, du travail collaboratif et de la gestion de projet. Elle englobe toutes les phases du cycle de développement logiciel : analyse des besoins, modélisation, implémentation, tests, documentation et présentation finale.

Dans ce contexte, nous avons choisi de développer une version numérique du **Plakoto**, un jeu traditionnel appartenant à la famille du backgammon. Ce choix s'est fondé sur les spécificités stratégiques du jeu, peu connues du grand public, mais offrant une richesse algorithmique intéressante à modéliser et à implémenter. Ce projet représente pour nous une opportunité concrète d'appliquer nos connaissances en Python.

3 Objectif

Compréhension approfondie du jeu Plakoto
- Étudier les règles officielles du jeu Plakoto, un jeu traditionnel de backgammon.
- Identifier les mécaniques clés : placement des pions, déplacements, captures, conditions de victoire, règles spéciales (comme la règle du "plakoto" qui consiste à bloquer un pion adverse).
- Analyser des parties existantes pour mieux appréhender la stratégie du jeu.

Modélisation des éléments du jeu
- Représenter le plateau de jeu de façon logique en Python (ex. : tableau, liste ou dictionnaire).
- Créer une structure de données pour gérer les pions de chaque joueur.
- Définir les états du jeu : début de partie, tour par tour, fin de partie.
- Gérer les aspects dynamiques du jeu, comme les déplacements et les blocages.

Implémentation technique en Python
- Utiliser des fonctions et des classes pour structurer proprement le code.
- Implémenter la logique de jeu : lancer de dés, alternance des tours, règles de déplacement, blocage de pions, victoire.
- Développer une interface textuelle ou graphique (selon la portée du projet).
- Gérer les entrées utilisateur, la validation des coups et les messages d'erreur.

Travail en équipe et compétences interpersonnelles
- Répartir efficacement les tâches entre les membres de l'équipe (développement, tests, documentation).
- Utiliser des outils de versionnage comme Git pour coordonner le travail.
- Maintenir une communication fluide (réunions régulières, partage des avancées, résolution des conflits).

Développement de compétences individuelles
- Renforcer sa maîtrise du langage Python et des principes de programmation orientée objet.
- Développer des capacités d'analyse et de résolution de problèmes liés à la logique de jeu.
- Améliorer son autonomie dans la recherche d'informations et le débogage.

Structuration d'un projet logiciel complet
- Organiser le code en modules clairs et réutilisables.
- Rédiger une documentation pour les fonctions et classes créées.
- Prévoir des tests pour vérifier le bon fonctionnement des règles et des cas limites.
- Réfléchir à une architecture évolutive permettant des améliorations futures (ex. : ajout d'une IA, interface graphique, mode multijoueur).

4 Conception des méthodes

Lors de ce projet, nous avons implémenter une interface textuelle, ainsi qu'une IA utilisant la méthode de Monte-Carlo. Pour cela, nous avons organisé notre travail en utilisant un “cycle en V”. Il est constitué de plusieurs étapes : Cahier des charges, Cahier des recettes, Conception détaillée, Développement, Intégration, recette et le rapport. De plus, lors du développement, nous avons procédé à de nombreux tests. Cela permet de garantir l'efficacité du programme.

4.1 Conception générale

Dans cette partie, nous allons nous intéresser au code du Plakoto, qui peut être séparé en plusieurs parties :

- **Affichage du plateau :** ces fonctions permettent d'initialiser et d'afficher le plateau
- **Déplacement des pions :** ces fonctions s'occupent de lancer les dés et de déplacer les pions, après avoir vérifié si le coup joué est possible.
- **Macro et Micro-coups :** ces fonctions créent une liste de tous les macro coups possibles, et leur micro-coups associés. Elles permettent de faire jouer les IA de manière optimisée.
- **Simulation :** ces fonctions servent à simuler une partie de Plakoto entre deux IA Monte-Carlo avec des niveaux personnalisables. Plus le niveau est élevé, plus chaque IA devra simuler des parties.
- **Autre :** ces fonctions ne peuvent pas faire partie d'une des autres classes de fonctions. Un exemple serait `gestion_commande`, qui permet de gérer les commandes entrées par l'utilisateur, et d'exécuter les fonctions nécessaires.

4.2 Conception détaillée de certaines fonctions importantes

Certaines fonctions sont particulièrement importantes quant à la bonne exécution du Plakoto. Voici certaines de ces fonctions, ainsi qu'une description de leur fonctionnement.

- **deplacer_pion()** : Cette fonction prend en entrée le plateau, l'origine, la destination et le joueur qui joue le pion bougé. Elle renvoie un boolean qui confirme le déplacement du pion.

  Elle vérifie si le joueur est autorisé à placer son pion sur la case de destination, puis si le pion essaye de sortir du plateau. Elle renvoie True si aucune des conditions spéciales n'ont été vérifiées.
- **generate_macro_coups()** : Elle prend en entrée le plateau, la liste des résultats des dés et le joueur qui doit jouer. Elle renvoie un dictionnaire qui contient une représentation du plateau comme clé, et l'ensemble des micro-coups jouables pour arriver à cette représentation comme valeur.

  Elle vérifie si la représentation du plateau est déjà une clé, dans ce cas elle renvoie la valeur associée à cette clé. Ensuite, elle initialise une pile qui contient la représentation du plateau, le joueur qui doit jouer pendant le tour, et les dés. Cela permet d'avoir une approche itérative, et non récursive. Tant que cette pile contient au moins un élément, elle essaye de jouer tous les dés restants sur tous les pions.
- **filtrer max pratique()** : Elle prend en entrée un dictionnaire qui contient une position et les macro-coups liés à cette position, et renvoie un dictionnaire des positions ainsi que leur macro-coups. Elle permet d'enlever tous les macro-coups qui n'atteignent pas le maximum pratique, c'est-à-dire le maximum de cases parcourues en un tour.
- **monte_carlo_select() :** elle prend en entrée le plateau, le joueur qui doit jouer, les dés, l'ensemble des macro-coups jouables, le nombre de simulations et le maximum de tours. Elle renvoie le meilleur macro-coup. Pour cela, elle lance autant de simulations de chaque macro-coup que nécessaire. Elle compte le nombre de

victoires de chaque macro-coup, et choisir celui qui a le plus de victoires. S'il y a une égalité, elle en choisit un au hasard.

5 Optimisations majeures

Au cours du développement du projet, plusieurs optimisations structurantes ont été introduites pour améliorer la robustesse, la performance et l'intelligence stratégique du système. Ces améliorations ont transformé un moteur de jeu interactif simple en une architecture modulaire, performante et capable de décisions autonomes. Les cinq grandes optimisations mises en œuvre sont les suivantes :

1. **Révision de la logique de validation des déplacements**

   Une nouvelle structure a été introduite, distinguant clairement les **micro-coups** (déplacements élémentaires) et les **macro-coups** (combinaisons complètes de déplacements pour un tour). Cela a permis une meilleure prise en compte de tous les chemins possibles et une validation rigoureuse des séquences de jeu, y compris les plus complexes.

2. **Ajout du mécanisme de bearing off et filtrage par efficacité maximale**

   Conformément aux règles du jeu (type Plakoto), un joueur peut désormais retirer ses pions une fois arrivés dans leur zone intérieure. Cette mécanique a été couplée à un système de **filtrage optimal** (appelé *max pratique*), permettant de ne conserver que les coups les plus efficaces selon les dés disponibles. Cette double vérification améliore la cohérence des décisions tout en renforçant l'adhérence aux règles stratégiques.

3. **Automatisation des parties aléatoires (IA vs IA)**

   Un simulateur complet a été mis en place pour permettre à deux IA de s'affronter automatiquement, sans intervention humaine. Chaque étape – du lancement des dés à la mise à jour du plateau – est automatisée, ce qui facilite la génération de données, l'observation des comportements à long terme et l'identification des faiblesses potentielles du système.

4. **Révision de la génération des macro-coups**

   Le processus de génération a été allégé et rendu plus efficace : les redondances ont été éliminées, les vérifications regroupées, et l'insertion des coups dans les structures de données a été optimisée. Cela a permis une meilleure utilisation de la mémoire, une réduction du nombre de coups générés, et une exécution plus rapide.

5. **Introduction du Monte Carlo et des simulations parallèles**

   Le simulateur Monte Carlo permet désormais à l'IA d'évaluer la qualité de chaque coup en simulant plusieurs parties possibles. Grâce au traitement parallèle, ces simulations sont effectuées plus rapidement, ce qui offre à l'IA une capacité d'anticipation stratégique bien plus élevée. L'IA ne se limite plus à des décisions immédiates, mais prend en compte les conséquences de ses choix sur plusieurs tours, améliorant ainsi significativement ses performances globales.

6 Résultats et analyse

6.1 Statistiques des parties IA vs IA

Pour évaluer les performances de notre IA Monte Carlo, nous avons réalisé une série de tests automatisés en mode "IA vs IA". Ces tests ont été effectués à l'aide de la méthode `match_ai_vs_ai()` dans un environnement entièrement contrôlé. Les simulations ont été exécutées en augmentant progressivement les valeurs de ns pour observer l'évolution des taux de victoire.

Chaque configuration a été testée sur 1000 ou 100 parties consécutives pour obtenir une vision claire de la répartition des victoires entre les deux IA. Les résultats montrent une nette progression des performances au fur et à mesure que le nombre de simulations augmente.

Avant d'analyser les résultats, il convient de préciser le cadre expérimental mis en place. Le paramètre principal étudié est **NS** (Number of Simulations), qui désigne le nombre d'itérations effectuées par la méthode Monte Carlo pour estimer la meilleure action à chaque coup. Il s'agit d'un facteur essentiel, car il influence directement la qualité des décisions prises par l'IA.

Dans le cadre de cette expérimentation, nous avons mené une série de tests appelés expérience du triplement, visant à évaluer l'impact d'un budget de simulation multiplié par trois entre deux IA en compétition (par exemple : 10 vs 30, 30 vs 100, 100 vs 300). L'objectif est de mieux comprendre à partir de quel seuil d'investissement computationnel les performances se stabilisent et les écarts de victoire s'atténuent.

6.2 Analyse des résultats des simulations Monte Carlo

Les résultats montrent une évolution progressive des taux de victoire en fonction de l'augmentation de ns. Dans les premières configurations, l'IA avec un nombre de simulations plus élevé obtient un avantage net, atteignant jusqu'à **67,8%** de taux de victoire.

Cependant, nous observons une phase de stabilisation dans la partie centrale de la courbe, lorsque les valeurs de ns augmentent (10 vs 30, 30 vs 100, 100 vs 300). Cette stabilisation s'explique par le fait que l'augmentation du budget de simulation des deux IA améliore simultanément leurs performances. En d'autres termes, l'amélioration n'est pas unilatérale : chaque IA devient plus performante, ce qui réduit l'écart de victoire et rapproche les taux vers un équilibre.

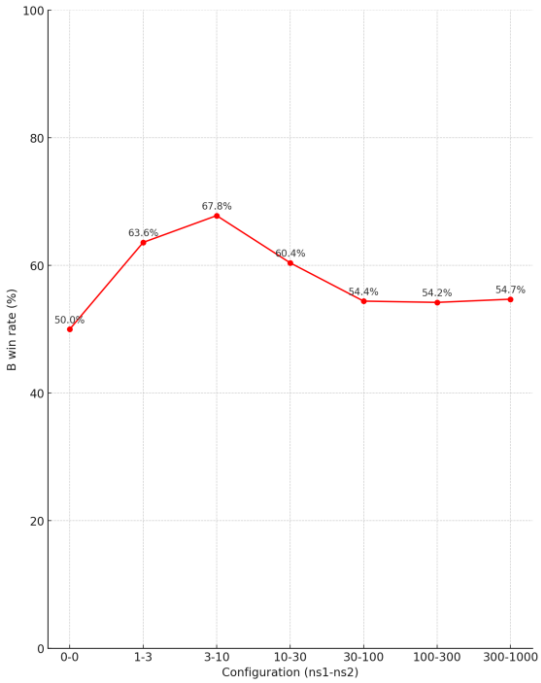
Cette situation illustre le phénomène de **saturation des échantillons**. À partir d'un certain point, l'ajout de simulations supplémentaires ne permet pas de découvrir de nouvelles stratégies, mais seulement de renforcer celles déjà connues. Cela réduit l'impact marginal de l'augmentation de ns sur le taux de victoire. C'est ce qui explique la légère diminution observée dans les configurations les plus élevées, où l'on converge vers un équilibre autour de **54%-55%**.

Ce comportement met en lumière un principe important :

— Lorsque les deux IA disposent de budgets élevés, l'avantage est minimisé car les stratégies atteignent un plafond d'efficacité.

ns1 (premier, N)	ns2 (second, B)	Nombre de parties	Taux de victoire B	Taux de victoire N	Taux de nulles
0	0	1000	50,0 %	50,0 %	4,8 %
1	3	1000	63,6 %	30,8 %	5,6 %
3	10	1000	67,8 %	32,2 %	6,6 %
10	30	1000	60,4 %	39,6 %	9,3 %
30	100	1000	54,4 %	45,6 %	8,2 %
100	300	300	54,2 %	45,8 %	9,0 %
300	1000	100	54,7 %	45,3 %	5,0 %

(a)



(b)

FIGURE 1 – Résultats des simulations Monte Carlo

— L'écart de performance dépend alors principalement de la différence relative entre les deux budgets, et non de leur valeur absolue.

Limites de l'expérimentation Bien que la courbe de taux de victoire tende à se stabiliser autour de **54–55%**, les derniers tests (par exemple **NS=300 vs NS=1000**) montrent que l'équilibre du jeu n'est pas encore totalement atteint.

En théorie, il serait nécessaire de continuer l'expérimentation avec des configurations plus extrêmes (comme **NS=1000 vs NS=3000** ou **NS=3000 vs NS=10000**) afin de déterminer avec précision à partir de quel budget "faible" les performances convergent véritablement vers 50%.

Cependant, nous avons choisi d'interrompre les tests à ce stade, pour deux raisons principales :

— **Durée d'exécution élevée** : Chaque configuration avancée requiert un temps de calcul significatif pour obtenir un nombre de parties suffisamment représentatif sur le plan statistique.

— **Coût en ressources de calcul** : Plus le nombre de simulations augmente, plus la charge processeur et mémoire devient importante, rendant ces tests coûteux en termes d'infrastructure.

6.3 Courbes de performance et interprétation

L'analyse des performances inclut également une évaluation du temps moyen de décision par coup en fonction de l'augmentation de `ns`. Les résultats montrent une augmentation quasi-linéaire du temps de calcul, avec un palier observé à partir de `ns = 500`. Cette analyse nous a permis de déterminer les configurations optimales pour les trois niveaux de difficulté de l'IA :

— **Easy** : ns = 30 — Décisions rapides et basiques
— **Medium** : ns = 500 — Bonne capacité de calcul et équilibre stratégie/temps
— **Hard** : ns = 1000 — Stratégie profonde avec un coût temporel accru

6.4 Explication des choix de ns et impact stratégique

Les tests effectués nous ont permis de constater l'importance du paramètre `ns` dans l'optimisation des décisions prises par l'IA Monte Carlo. Nous avons constaté que les paliers à 30, 500 et 1000 représentent les meilleurs compromis entre le temps de calcul et l'efficacité stratégique.

— À faible coût (`ns = 30`), l'IA effectue des choix simples mais cohérents.
— À coût moyen (`ns = 500`), l'IA peut anticiper des mouvements complexes tout en restant dans un temps acceptable.
— À coût élevé (`ns = 1000`), les décisions sont quasi optimales, mais le temps de réponse augmente de manière significative.

Ainsi, l'ajustement de ns permet de moduler le niveau de difficulté et de garantir une expérience de jeu équilibrée et progressive.

7 Déroulement du développement

Le développement du projet s'est articulé autour de trois grandes phases progressives, s'étalant sur dix semaines, depuis la définition des objectifs jusqu'à la mise en place d'une intelligence artificielle de décision.

Phase 1 – Analyse des besoins et conception initiale (Semaines 1 à 3) Cette première étape a consisté en une étude approfondie des règles du jeu *Plakoto*, afin d'évaluer leur transposition en logique de programmation.

Une analyse fonctionnelle a permis de dégager les besoins clés :
— Affichage du plateau de jeu,
— Réalisation des déplacements,
— Détection des conditions de victoire,
— Gestion de règles spéciales (ex. bearing off).

Le tout a été formalisé dans un cahier des charges et un cahier de recette, accompagnés de la conception d'une architecture de base avec interface textuelle et moteur de jeu.

Phase 2 – Développement du moteur de jeu et interface (Semaines 4 à 6) La seconde phase a porté sur l'implémentation concrète du plateau et de l'interface textuelle :
— Initialisation du plateau et affichage des états de jeu,
— Création d'une interface de saisie permettant la validation des coups en temps réel,
— Mise en place d'un système de commande interprétant les actions du joueur.

Des choix de design ont été faits pour optimiser l'ergonomie : seuls les coups saisis par l'utilisateur sont validés, évitant ainsi une surcharge visuelle.

Enfin, une première série de tests complets a permis de valider les déplacements, les conditions de victoire, ainsi que les cas limites.

Phase 3 – Intégration de l'IA Monte Carlo et simulateur (Semaines 7 à 10) La dernière étape du développement a introduit une IA décisionnelle basée sur le principe du Monte Carlo :

— Génération et optimisation des coups possibles avec mise en cache pour limiter les calculs redondants,
— Simulation d'un grand nombre de parties pour évaluer les choix stratégiques,
— Exécution parallèle des simulations grâce à `ThreadPoolExecutor` pour accélérer le processus.

Des tests automatiques ont été menés (IA vs IA) à l'aide de différents paramètres, permettant d'évaluer l'efficacité de l'IA selon une méthode comparative appelée "triplens".

Phase 4 – Finalisation de la documentation et optimisation du code (Semaines 11 à 12) La dernière phase du projet a été consacrée à la finalisation du travail sous deux aspects principaux :

— Rédaction et structuration du mémoire de projet, incluant la description des différentes phases de développement, les choix techniques justifiés, ainsi que les résultats obtenus
— Nettoyage et optimisation du code source afin d'améliorer sa lisibilité, sa modularité et ses performances.

Un effort particulier a été apporté à la clarté et à la cohérence de la documentation, dans le but de rendre le projet compréhensible et réutilisable pour un tiers. Cette étape a permis de clôturer le développement dans des conditions de qualité et de professionnalisme.

8 Discussion collective

8.1 Organisation et gestion de projet

Pendant tout le projet, nous avons suivi un planning précis pour avancer étape par étape. Chaque semaine, nous faisions le point pour savoir où nous en étions et pour ajuster notre travail si besoin. Nous utilisions un outil (TortoiseSVN) pour partager et versionner notre code : chacun travaillait sur sa branche, ce qui évitait les conflits et permettait de revenir en arrière facilement. Les réunions régulières et les revues de code nous ont aidés à corriger rapidement les erreurs et à garder un logiciel de bonne qualité. Cette organisation nous a permis de terminer les différentes étapes (conception, développement, tests, documentation) dans les temps et de façon claire pour tous.

8.2 Travail en équipe et communication

Le vrai point fort de notre groupe a été la façon dont nous avons communiqué et travaillé ensemble. Chaque jour, nous partagions nos avancées, nos difficultés et nos idées lors de courts “stand-up” ou par messages écrits. Quand un désaccord arrivait (par exemple, sur l'apparence de l'interface ou sur la méthode de test), nous mettions en place de petits essais concrets pour décider : un prototype, un essai rapide, un partage de résultat. Cela nous a permis de prendre des décisions justes et d'apprendre les uns des autres. Pour préparer le mémoire et la soutenance, nous avons travaillé par binômes, relu nos textes mutuellement et fait des simulations d'oral. Grâce à cette entraide, nous avons gagné en confiance et en fluidité dans nos présentations.

8.3 Réussites et apprentissages

Ce projet nous a permis de livrer une version opérationnelle et fiable de Plakoto, avec une interface textuelle et une IA Monte Carlo efficace grâce à un cache global et à la parallélisation. Le respect du cycle en V et les tests automatisés ont assuré la stabilité du code. Nous avons compris que la mémoïsation et le traitement parallèle offrent de gros gains de performance, mais requièrent une gestion rigoureuse de l'état et des ressources. Par ailleurs, la documentation régulière et la communication entre membres se sont avérées essentielles pour garder tous les acteurs alignés. Enfin, la mise en place de benchmarks clairs nous a aidés à mesurer l'impact de chaque optimisation et à faire des choix techniques éclairés.

9 Conclusion

9.1 Bilan et synthèse

Dans ce mémoire, nous avons retracé l'ensemble de notre travail pour développer une version numérique du jeu Plakoto. Nous avons commencé par étudier en détail les règles du jeu et rédiger un cahier des charges précis, afin de poser une base solide pour la conception. La modélisation du plateau et des pions sous forme de structures Python simples nous a permis de rapidement mettre en place les premières fonctionnalités : affichage du plateau, déplacement des pions et gestion des règles spéciales comme le « bearing off ». Ces premières étapes ont servi à valider notre compréhension du jeu et à établir le socle technique sur lequel s'appuyer.

Ensuite, nous avons enrichi ce socle en introduisant la génération de macro-coups et la simulation de micro-coups. Grâce à une approche itérative et à l'usage d'un cache global (macro-moves_cache), nous avons limité les recalculs redondants et gagné en efficacité. L'ajout d'une intelligence artificielle basée sur Monte Carlo, parallélisée avec ProcessPoolExecutor, a constitué l'étape clé du projet. Cette IA simule plusieurs parties aléatoires pour évaluer la qualité des coups, et nous avons mis en place un protocole d'expérimentation permettant d'observer l'influence du nombre de simulations (ns) sur le taux de victoire et le temps de calcul.

Nos tests, menés sur plusieurs séries de parties IA vs IA, ont clairement montré qu'au-delà d'un certain seuil de simulations, l'amélioration du taux de victoire devient très faible, alors que le temps de calcul augmente sensiblement. Ces résultats nous ont guidés pour proposer une valeur de ns « pratique » à utiliser, équilibrant précision des décisions et contraintes de temps. Par ailleurs, l'usage d'un cycle en V, complété par des revues de code et des tests automatisés, a renforcé la qualité et la stabilité du programme à chaque étape.

9.2 Perspectives et améliorations

Une des évolutions les plus naturelles serait de doter Plakoto d'une interface graphique moderne, plutôt que de l'affichage ASCII. On pourrait imaginer une fenêtre simple avec Tkinter ou PyQt, où l'on glisse les pions et clique sur un bouton pour lancer les dés. Cela rendrait le jeu plus intuitif, notamment pour les nouveaux joueurs. Plus ambitieux, une version web avec Flask en backend et un petit framework JavaScript (Vue ou React) offrirait un accès instantané depuis un navigateur, sans installation.

Côté intelligence artificielle, l'utilisation d'un Monte Carlo Tree Search (MCTS) pourrait améliorer la qualité des coups pour le même temps de calcul. Au lieu de lancer N parties entièrement aléatoires, MCTS construit progressivement un arbre de décisions, en privilégiant les branches les plus prometteuses. À terme, on pourrait même explorer l'apprentissage par renforcement : l'IA apprendrait en jouant contre elle-même, sans intervention humaine, pour affiner ses évaluations de position.

Enfin, pour simplifier l'installation et le paramétrage, on pourrait créer un paquet Python installable (via pip) ou un exécutable multiplateforme (avec PyInstaller). Un petit menu de réglages en début de partie permettrait de choisir le niveau de l'IA, le temps maximum par coup ou les heuristiques à activer, sans modifier le code. Ces améliorations rendraient Plakoto plus accessible et plus complet, tout en offrant de nouvelles pistes de recherche pour l'IA et l'expérience utilisateur.